

Tarea – Introducción a Python, R y Julia

Viviana Andrea Peña González

Table of contents

1	Introducción	1
2	Ejercicio 1. Hidrología del Río Magdalena	2
2.0.1	¿Qué sucedería si este ejercicio se hiciera en Python usando una lista nativa y se multiplicara directamente por 1000?	3
3	Ejercicio 2. Calidad del aire en Bogotá	3
3.0.1	Escribe la línea de código exacta que usarías en R para extraer las tres primeras estaciones del <code>data.frame</code> del ejercicio 2 usando indexación por rangos. Explica brevemente si el límite superior del rango se incluye o se excluye en el resultado final.	6
4	Conclusión	6

1 Introducción

En este taller se trabajan operaciones introductorias con vectores y tablas de datos en un contexto aplicado a problemas ambientales y geográficos. En particular, se abordan estadísticos básicos, vectorización de operaciones numéricas, creación de `data.frame`, filtrado por condiciones y visualización simple de datos.

Aunque el capítulo presenta una comparación general entre Python, R y Julia, en esta resolución se utiliza R como lenguaje principal. Esto permite resolver los ejercicios mediante operaciones vectorizadas nativas y estructuras tabulares sencillas, útiles para el análisis inicial de datos hidrológicos y atmosféricos.

2 Ejercicio 1. Hidrología del Río Magdalena

En este ejercicio se analizan los caudales medios mensuales de cinco estaciones del Río Magdalena. El enunciado pide crear dos colecciones paralelas, calcular el valor máximo y el promedio, y luego transformar las unidades de metros cúbicos por segundo a litros por segundo utilizando vectorización y sin usar ciclos `for`.

```
cat("---- Ejercicio 1: Hidrología del Río Magdalena ----\n")
```

---- Ejercicio 1: Hidrología del Río Magdalena ----

```
# 1. Colección de estaciones
estaciones <- c(
  "Honda",
  "Puerto Berrío",
  "Barrancabermeja",
  "Puerto Wilches",
  "Calamar"
)

# 2. Colección paralela de caudales en m3/s
caudales_m3s <- c(1500, 2100, 2800, 3200, 4500)

# 3. Estadísticos básicos
caudal_maximo <- max(caudales_m3s)
caudal_promedio <- mean(caudales_m3s)

cat("Caudal máximo registrado:", caudal_maximo, "m3/s\n")
```

Caudal máximo registrado: 4500 m3/s

```
cat("Caudal promedio:", caudal_promedio, "m3/s\n")
```

Caudal promedio: 2820 m3/s

```
# 4. Vectorización: conversión a litros por segundo
caudales_ls <- caudales_m3s * 1000

# 5. Imprimir resultado
cat("Caudales en litros por segundo:\n")
```

Caudales en litros por segundo:

```
print(caudales_ls)
```

```
[1] 1500000 2100000 2800000 3200000 4500000
```

En R, este ejercicio se resuelve de forma directa porque los vectores soportan operaciones vectorizadas de manera nativa. Esto significa que al multiplicar `caudales_m3s * 1000`, R aplica la operación a todos los elementos del vector sin necesidad de iteraciones explícitas.

! Resultado del ejercicio 1

La ejecución del código permitió obtener un caudal máximo de 4500 m³/s y un caudal promedio de 2820 m³/s. Además, la conversión vectorizada produjo la colección 1500000, 2100000, 2800000, 3200000 y 4500000 l/s.

i Sobre el ejercicio 1

2.0.1 ¿Qué sucedería si este ejercicio se hiciera en Python usando una lista nativa y se multiplicara directamente por 1000?

Si en Python se crea `caudales_m3s` como una lista nativa, la operación `caudales_m3s * 1000` no realizaría la multiplicación matemática elemento por elemento. En lugar de eso, Python repetiría la lista mil veces, porque ese es el comportamiento estándar de las listas al multiplicarlas por un entero.

La librería que soluciona este problema es NumPy, ya que sus arreglos sí permiten operaciones vectorizadas reales. La diferencia frente a R es que en R la vectorización es nativa para estructuras como `c()`, mientras que en Julia este comportamiento también existe, aunque normalmente se hace explícito mediante broadcasting cuando corresponde. Por eso, en R este ejercicio puede resolverse directamente con vectores base sin necesidad de una librería adicional.

3 Ejercicio 2. Calidad del aire en Bogotá

En este ejercicio se construye un `data.frame` con estaciones de monitoreo de calidad del aire en Bogotá y sus niveles promedio diarios de PM2.5. Luego se imprime un resumen técnico, se filtran las estaciones con valores estrictamente mayores a 15, se crea una columna llamada `estado` con el valor "Crítico" y finalmente se genera un gráfico de barras usando el `data.frame` original.

```
cat("---- Ejercicio 2: Calidad del Aire en Bogotá ----\n")
```

---- Ejercicio 2: Calidad del Aire en Bogotá ----

```
# 1. Crear DataFrame
df_aire <- data.frame(
  estacion = c("Carvajal", "Kennedy", "Fontibón", "Suba", "Usaquén"),
  pm25 = c(55, 42, 38, 15, 12)
)

# 2. Resumen descriptivo/técnico
cat("Estructura del DataFrame:\n")
```

Estructura del DataFrame:

```
str(df_aire)
```

```
'data.frame':  5 obs. of  2 variables:
 $ estacion: chr  "Carvajal" "Kennedy" "Fontibón" "Suba" ...
 $ pm25    : num  55 42 38 15 12
```

```
# 3. Filtrar estaciones con PM2.5 > 15
df_alerta <- df_aire[df_aire$pm25 > 15, ]

# 4. Crear nueva columna
df_alerta$estado <- "Crítico"

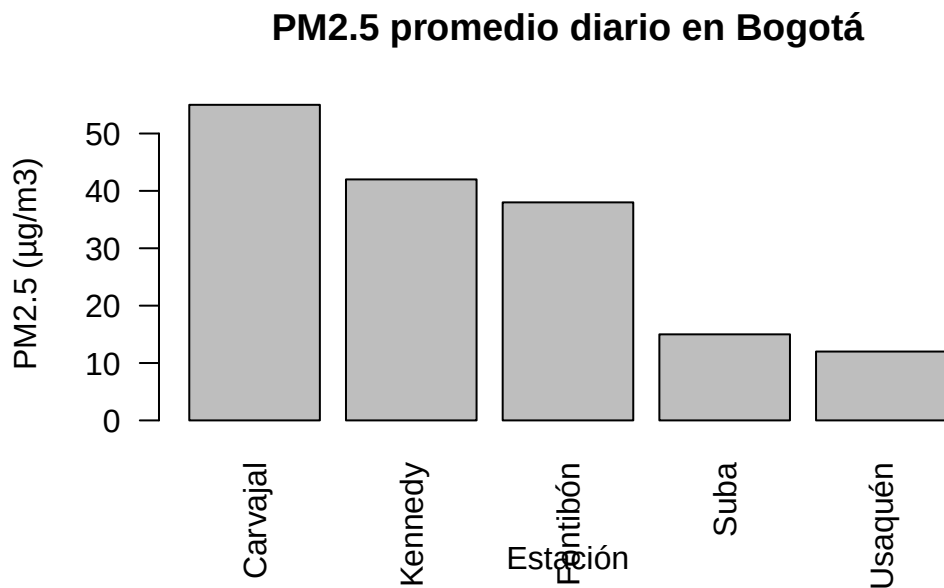
cat("\nEstaciones en alerta:\n")
```

Estaciones en alerta:

```
print(df_alerta)
```

```
  estacion pm25 estado
1 Carvajal   55 Crítico
2 Kennedy   42 Crítico
3 Fontibón  38 Crítico
```

```
# 5. Gráfico de barras
barplot(
  df_aire$pm25,
  names.arg = df_aire$estacion,
  main = "PM2.5 promedio diario en Bogotá",
  xlab = "Estación",
  ylab = "PM2.5 (µg/m3)",
  las = 2
)
```



Aquí se utilizó `data.frame()` para estructurar los datos tabulares y `str()` para inspeccionar la estructura del objeto, lo que equivale a un resumen técnico básico. Después, el filtrado por condición permitió aislar las estaciones que superan el límite recomendado por la OMS.

! Resultado del ejercicio 2

El filtrado generó un nuevo `data.frame` con las estaciones Carvajal, Kennedy y Fontibón, ya que son las únicas con niveles de PM2.5 estrictamente mayores a 15. A todas ellas se les asignó correctamente el estado **Crítico**.

i Pregunta general sobre indexación

3.0.1 Escribe la línea de código exacta que usarías en R para extraer las tres primeras estaciones del `data.frame` del ejercicio 2 usando indexación por rangos. Explica brevemente si el límite superior del rango se incluye o se excluye en el resultado final.

La línea de código exacta en R sería:

```
df_aire[1:3, ]
```

En este caso, el límite superior del rango sí se incluye. Esto ocurre porque R utiliza indexación base 1 y, además, los rangos como `1:3` abarcan todos los enteros entre ambos extremos. Por tanto, el resultado contiene exactamente la primera, segunda y tercera fila del `data.frame`.

4 Conclusión

Este taller permitió reconocer diferencias fundamentales entre Python, R y Julia en tareas básicas de geocomputación, especialmente en el manejo de vectores, tablas e indexación. También mostró que operaciones aparentemente simples, como multiplicar una colección o extraer un subconjunto de filas, pueden comportarse de forma distinta según el lenguaje.

Desde una perspectiva aplicada, los ejercicios permiten conectar la programación introductoria con problemas reales de geomática, como el análisis de caudales, la organización de estaciones de monitoreo y la evaluación de umbrales ambientales.